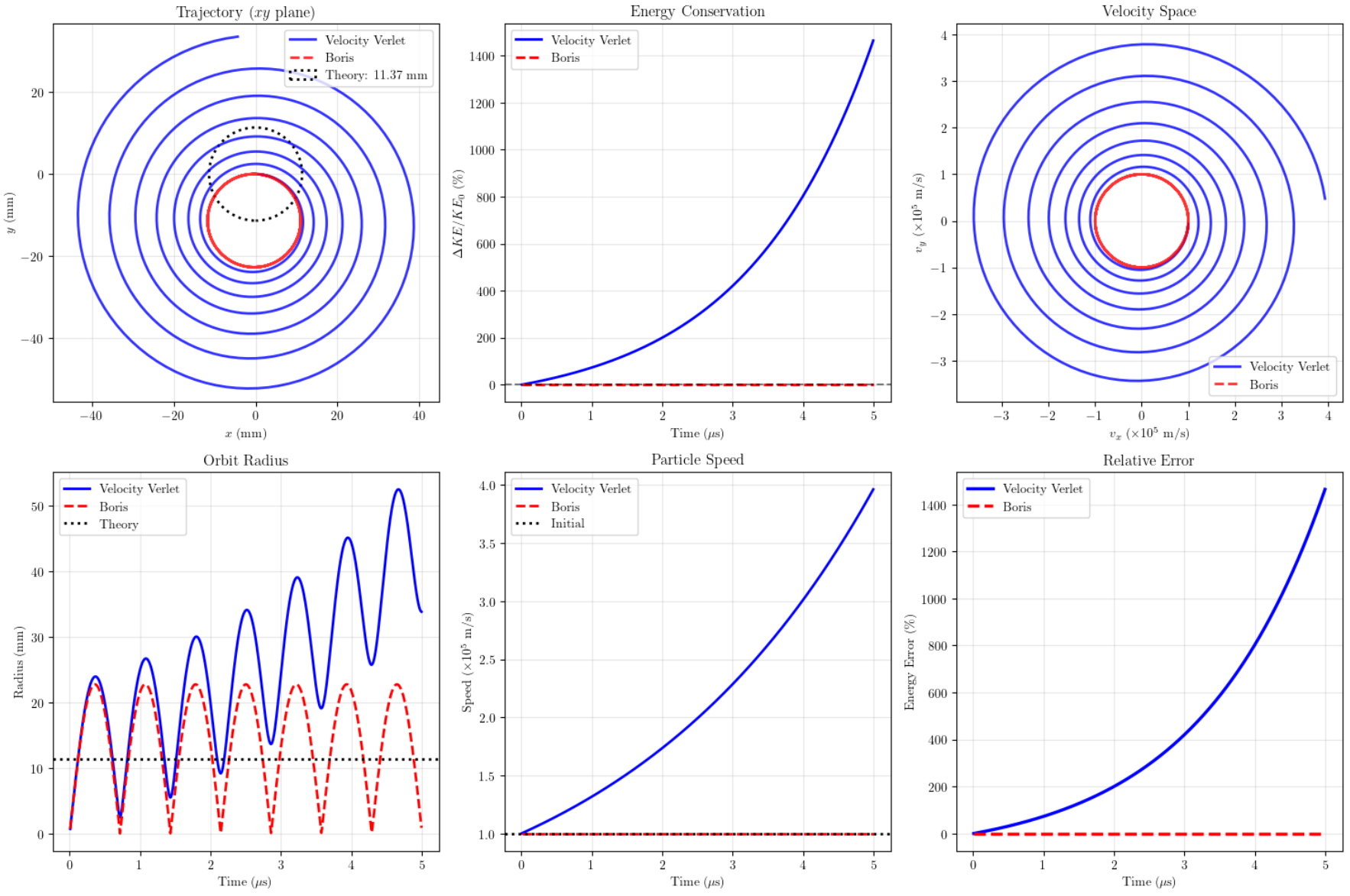
Motion of charged particles in a homogeneous stationary electromagnetic field

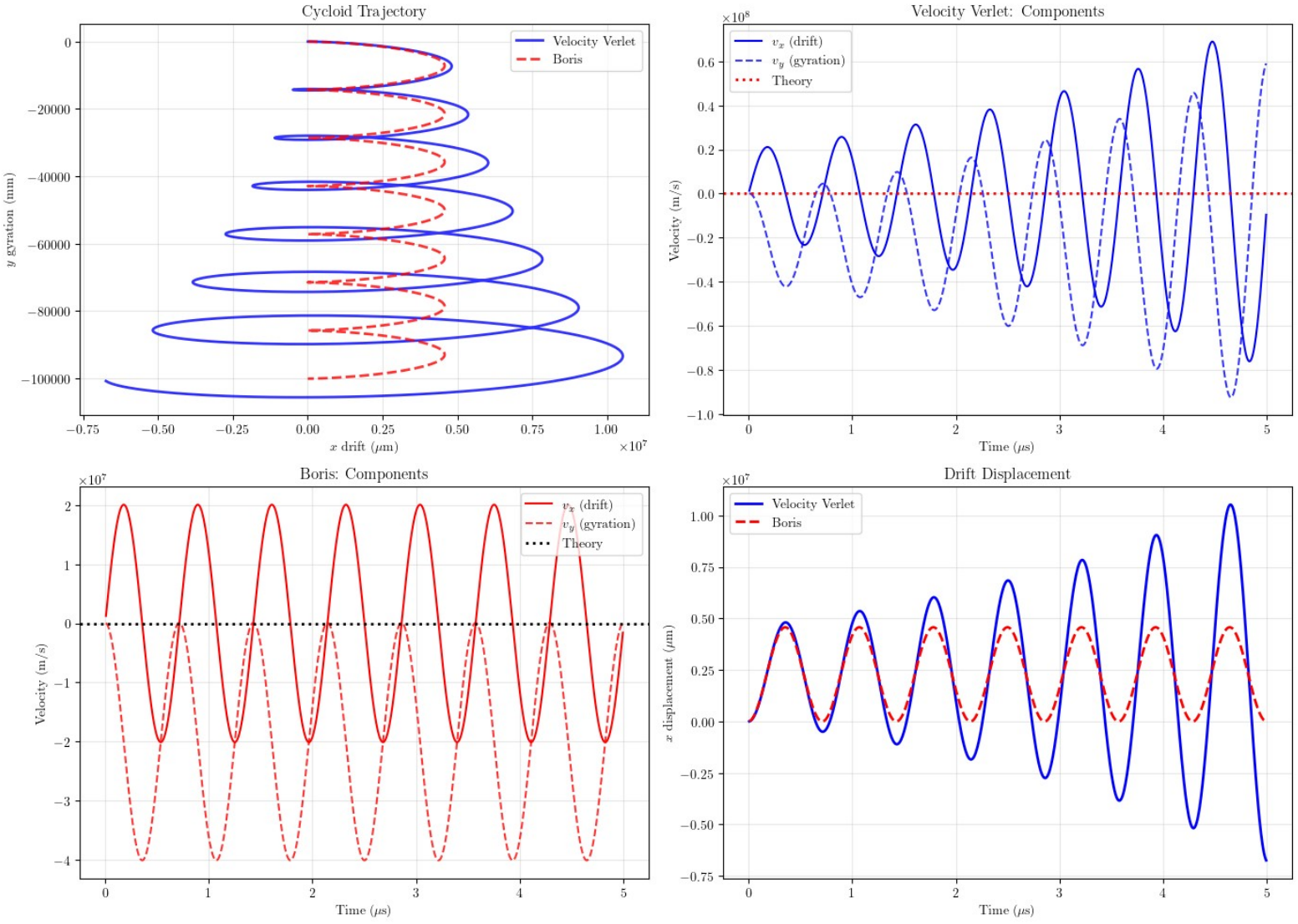
Presented by: Sekh Mohammed Mohasin

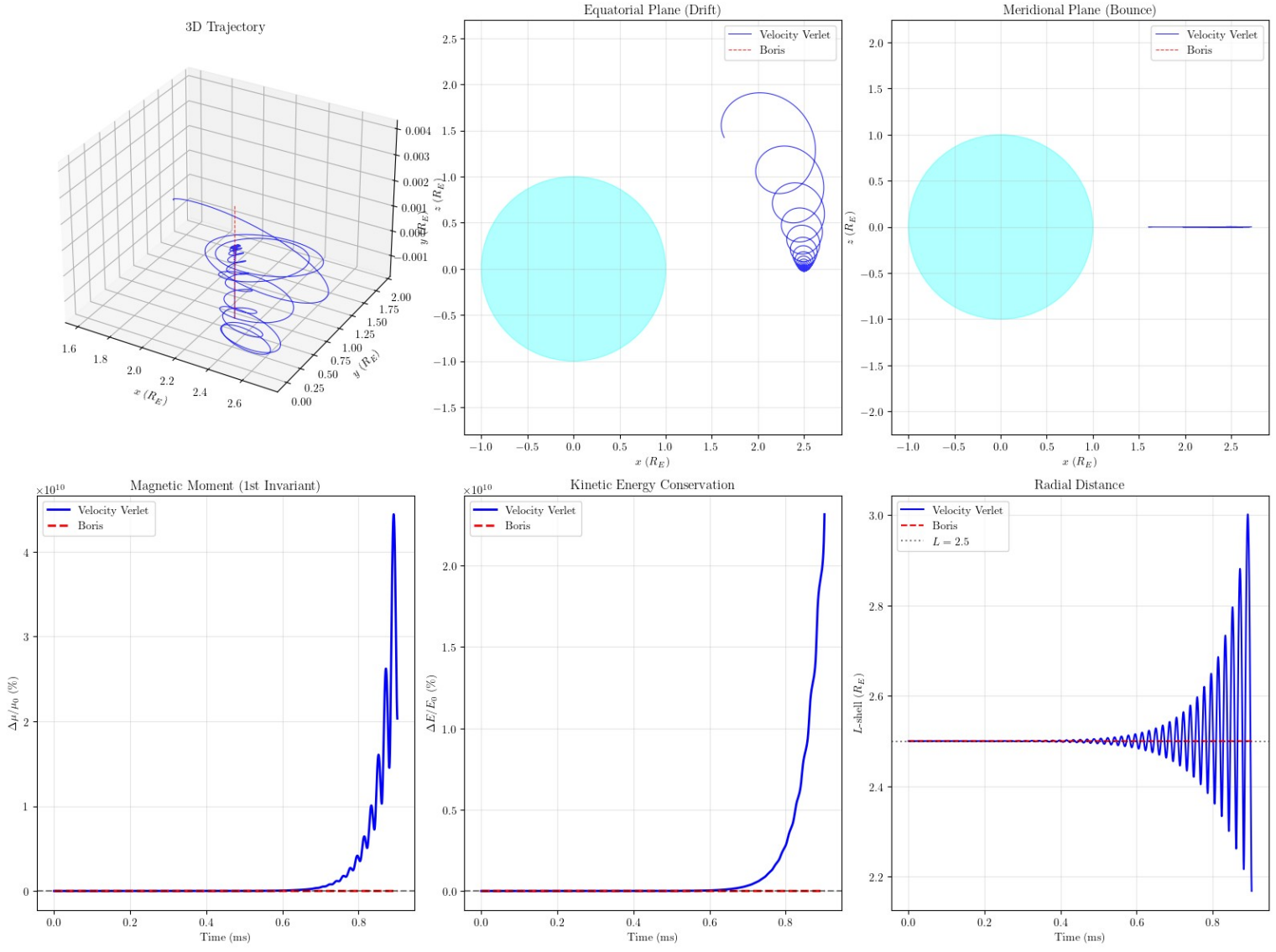
Master 2 SSA – Fundamental Physics and Application (2025–2026)

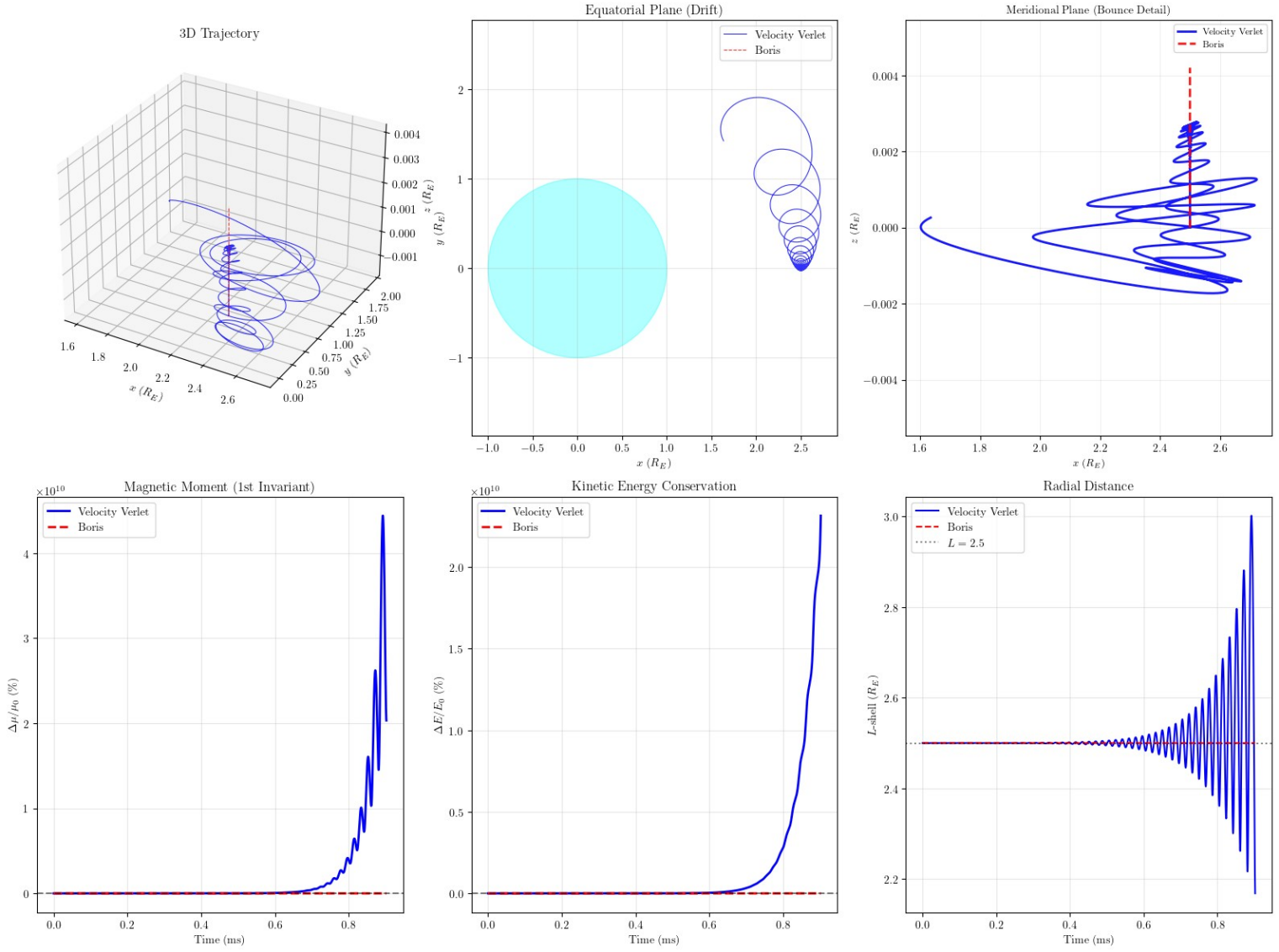
N° Étudiant: 22508342

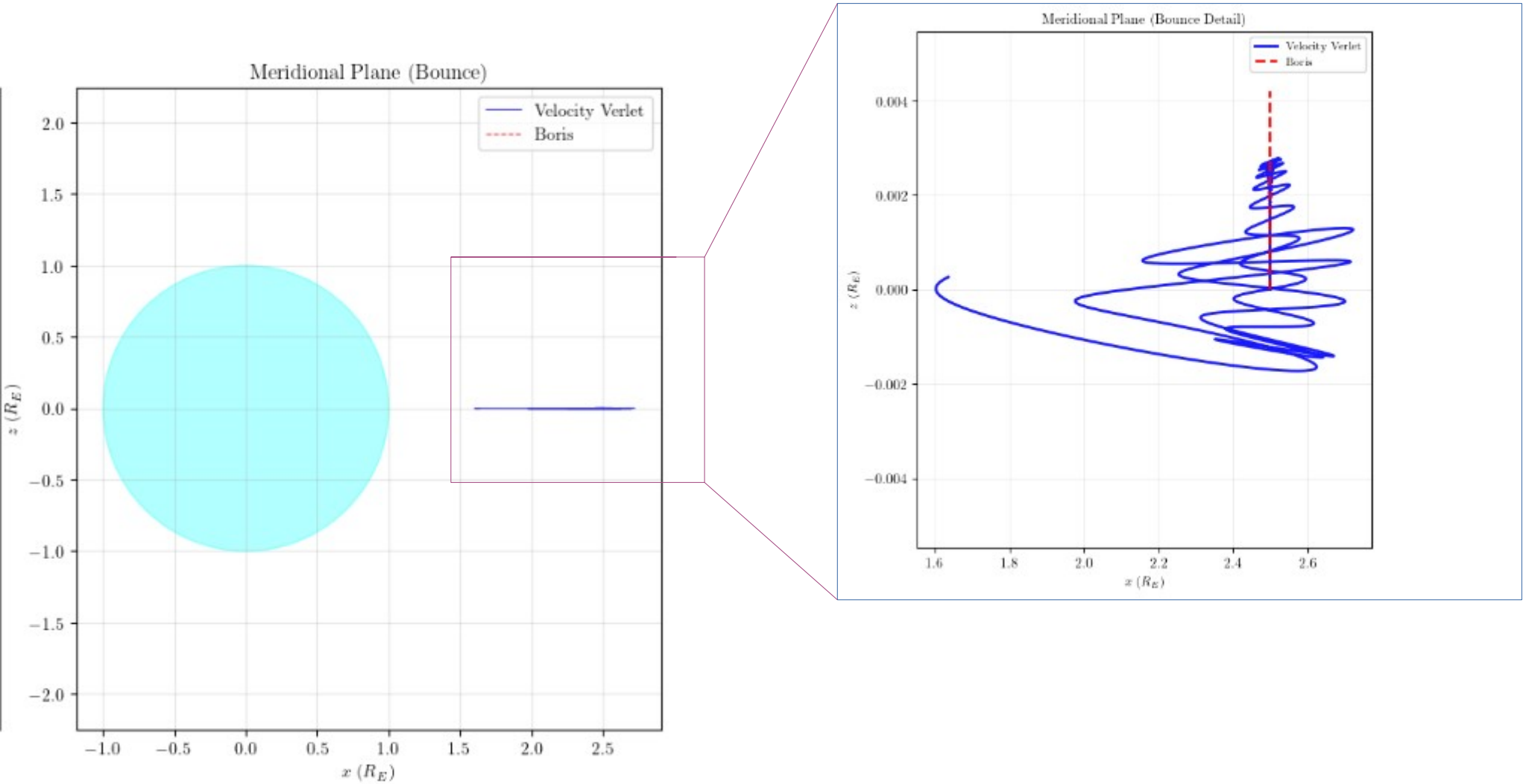
Université d'ORLÉANS


Trajectory (xy plane)
Velocity Verlet
Boris
Theory: 11.37 mm
y (mm)
x (mm)
Energy Conservation
Velocity Verlet
Boris
$\Delta KE/KE_0$ (%)
Time (μs)
Velocity Space
Velocity Verlet
Boris
v_y ($\times 10^5$ m/s)
v_x ($\times 10^5$ m/s)
Orbit Radius
Velocity Verlet
Boris
Theory
Radius (mm)
Time (μs)
Particle Speed
Velocity Verlet
Boris
Initial
Speed ($\times 10^5$ m/s)
Time (μs)
Relative Error
Velocity Verlet
Boris
Energy Error (%)
Time (μs)


Cycloid Trajectory
Velocity Verlet
Boris
y gyration (mm)
x drift (μm)
×10^7
Velocity Verlet: Components
×10^8
v_x (drift)
v_y (gyration)
Theory
Velocity (m/s)
Time (μs)
Boris: Components
×10^7
v_x (drift)
v_y (gyration)
Theory
Velocity (m/s)
Time (μs)
Drift Displacement
×10^7
Velocity Verlet
Boris
x displacement (μm)
Time (μs)


3D Trajectory
x (R_E)
y (R_E)
z (R_E)
Equatorial Plane (Drift)
Velocity Verlet
Boris
x (R_E)
z (R_E)
Meridional Plane (Bounce)
Velocity Verlet
Boris
x (R_E)
z (R_E)
Magnetic Moment (1st Invariant)
×10^10
Velocity Verlet
Boris
Δμ/μ0 (%)
Time (ms)
Kinetic Energy Conservation
×10^10
Velocity Verlet
Boris
ΔE/E0 (%)
Time (ms)
Radial Distance
Velocity Verlet
Boris
L = 2.5
L-shell (R_E)
Time (ms)


3D Trajectory
$x\ (R_E)$
$y\ (R_E)$
$z\ (R_E)$
Equatorial Plane (Drift)
Velocity Verlet
Boris
$x\ (R_E)$
$y\ (R_E)$
Meridional Plane (Bounce Detail)
Velocity Verlet
Boris
$x\ (R_E)$
$z\ (R_E)$
Magnetic Moment (1st Invariant)
$\times 10^{10}$
Velocity Verlet
Boris
$\Delta\mu/\mu_0$ (%)
Time (ms)
Kinetic Energy Conservation
$\times 10^{10}$
Velocity Verlet
Boris
$\Delta E/E_0$ (%)
Time (ms)
Radial Distance
Velocity Verlet
Boris
$L = 2.5$
L-shell (R_E)
Time (ms)


Meridional Plane (Bounce)
Velocity Verlet
Boris
2.0
1.5
1.0
0.5
0.0
−0.5
−1.0
−1.5
−2.0
$z\ (R_E)$
−1.0
−0.5
0.0
0.5
1.0
1.5
2.0
2.5
$x\ (R_E)$
Meridional Plane (Bounce Detail)
Velocity Verlet
Boris
0.004
0.002
0.000
−0.002
−0.004
$z\ (R_E)$
1.6
1.8
2.0
2.2
2.4
2.6
$x\ (R_E)$

Python Code

```python

import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D

# Enable LaTeX rendering
plt.rcParams['text.usetex'] = True
plt.rcParams['font.family'] = 'serif'
plt.rcParams['text.latex.preamble'] = r'\usepackage{amsmath}'

# Physical constants
q_e = 1.602e-19
m_e = 9.109e-31
m_p = 1.673e-27
c = 3e8
RE = 6.371e6
DIPOLE_MOMENT = 8.0e22



def velocity_verlet(B, v0, dt, t_final, E=None):


    if E is None:
        E = np.array([0.0, 0.0, 0.0])

    nsteps = int(t_final / dt)

    r = np.array([0.0, 0.0, 0.0])
    v = np.array([v0, 0.0, 0.0])

    positions = np.zeros((nsteps, 3))
    velocities = np.zeros((nsteps, 3))
    times = np.zeros(nsteps)
    kinetic_energies = np.zeros(nsteps)

    for i in range(nsteps):
        # Half-step position
        r_half = r + 0.5 * dt * v

        # Acceleration (Lorentz force)
        a = (q_e / m_e) * (np.cross(v, B) + E)

        # Full-step velocity
        v_new = v + dt * a

        # Full-step position
        r_new = r_half + 0.5 * dt * v_new

        r = r_new
```

```python
        v = v_new

        positions[i] = r
        velocities[i] = v
        times[i] = (i + 1) * dt
        kinetic_energies[i] = 0.5 * m_e * np.sum(v**2)

    return times, positions, velocities, kinetic_energies


#  BORIS ALGORITHM


def boris_algorithm(B, v0, dt, t_final, E=None):

    if E is None:
        E = np.array([0.0, 0.0, 0.0])

    nsteps = int(t_final / dt)

    r = np.array([0.0, 0.0, 0.0])
    v = np.array([v0, 0.0, 0.0])

    positions = np.zeros((nsteps, 3))
    velocities = np.zeros((nsteps, 3))
    times = np.zeros(nsteps)
    kinetic_energies = np.zeros(nsteps)

    for i in range(nsteps):
        # Half-step E field
        v_minus = v + 0.5 * (q_e / m_e) * E * dt

        # Exact magnetic rotation
        t_vec = 0.5 * (q_e / m_e) * B * dt
        t_sq = np.dot(t_vec, t_vec)
        s_vec = 2 * t_vec / (1 + t_sq)

        v_prime = v_minus + np.cross(v_minus, t_vec)
        v_plus = v_minus + np.cross(v_prime, s_vec)

        # Half-step E field
        v = v_plus + 0.5 * (q_e / m_e) * E * dt

        # Position update
        r = r + v * dt

        positions[i] = r
        velocities[i] = v
        times[i] = (i + 1) * dt
        kinetic_energies[i] = 0.5 * m_e * np.sum(v**2)

    return times, positions, velocities, kinetic_energies


```

```python


# EXB DRIFT WITH BOTH ALGORITHMS



def velocity_verlet_fields(B, E, v_initial, dt, t_final, mass=m_e, charge=q_e):

    nsteps = int(t_final / dt)

    r = np.array([0.0, 0.0, 0.0])
    v = v_initial.copy()

    positions = np.zeros((nsteps, 3))
    velocities = np.zeros((nsteps, 3))
    times = np.zeros(nsteps)

    for i in range(nsteps):
        r_half = r + 0.5 * dt * v
        a = (charge / mass) * (np.cross(v, B) + E)
        v_new = v + dt * a
        r_new = r_half + 0.5 * dt * v_new

        r = r_new
        v = v_new

        positions[i] = r
        velocities[i] = v
        times[i] = (i + 1) * dt

    return times, positions, velocities



def boris_fields(B, E, v_initial, dt, t_final, mass=m_e, charge=q_e):

    nsteps = int(t_final / dt)

    r = np.array([0.0, 0.0, 0.0])
    v = v_initial.copy()

    positions = np.zeros((nsteps, 3))
    velocities = np.zeros((nsteps, 3))
    times = np.zeros(nsteps)

    for i in range(nsteps):
        v_minus = v + 0.5 * (charge / mass) * E * dt

        t_vec = 0.5 * (charge / mass) * B * dt
        t_sq = np.dot(t_vec, t_vec)
        s_vec = 2 * t_vec / (1 + t_sq)

        v_prime = v_minus + np.cross(v_minus, t_vec)
        v_plus = v_minus + np.cross(v_prime, s_vec)
```

```python

        v = v_plus + 0.5 * (charge / mass) * E * dt
        r = r + v * dt

        positions[i] = r
        velocities[i] = v
        times[i] = (i + 1) * dt

    return times, positions, velocities



# DIPOLE FIELD



def dipole_field(r, M=DIPOLE_MOMENT):

    r_norm = np.linalg.norm(r)

    if r_norm < 1e-10:
        return np.array([0.0, 0.0, 0.0])

    m = np.array([0.0, 0.0, 1.0])
    r_hat = r / r_norm

    B = 1e-7 * M * (3.0 * np.dot(m, r_hat) * r_hat - m) / r_norm**3

    return B



def simulate_dipole_verlet(particle_mass, particle_charge, r0, v0, dt, t_final):

    nsteps = int(t_final / dt)

    r = r0.copy()
    v = v0.copy()

    positions = []
    velocities = []
    B_fields = []
    times = []

    for i in range(nsteps):
        r_mag = np.linalg.norm(r)

        if r_mag < RE or r_mag > 10 * RE:
            break

        B = dipole_field(r)

        r_half = r + 0.5 * dt * v
```

```python
        a = (particle_charge / particle_mass) * np.cross(v, B)
        v_new = v + dt * a
        r_new = r_half + 0.5 * dt * v_new

        r = r_new
        v = v_new

        positions.append(r.copy())
        velocities.append(v.copy())
        B_fields.append(B.copy())
        times.append((i + 1) * dt)

    return (np.array(times), np.array(positions),
            np.array(velocities), np.array(B_fields))



def simulate_dipole_boris(particle_mass, particle_charge, r0, v0, dt, t_final):

    nsteps = int(t_final / dt)

    r = r0.copy()
    v = v0.copy()

    positions = []
    velocities = []
    B_fields = []
    times = []

    for i in range(nsteps):
        r_mag = np.linalg.norm(r)

        if r_mag < RE or r_mag > 10 * RE:
            break

        B = dipole_field(r)

        t_vec = 0.5 * (particle_charge / particle_mass) * B * dt
        s_vec = 2 * t_vec / (1 + np.dot(t_vec, t_vec))

        v_prime = v + np.cross(v, t_vec)
        v = v + np.cross(v_prime, s_vec)

        r = r + v * dt

        positions.append(r.copy())
        velocities.append(v.copy())
        B_fields.append(B.copy())
        times.append((i + 1) * dt)

    return (np.array(times), np.array(positions),
            np.array(velocities), np.array(B_fields))

```

```python



# PLOTS


def plot_cyclotron_motion(B_mag=50e-6, v0=1e5, t_final=5e-6):

    B = np.array([0, 0, B_mag])

    omega_c = abs(q_e) * B_mag / m_e
    T_c = 2 * np.pi / omega_c
    dt = T_c / 100
    r_L = m_e * v0 / (abs(q_e) * B_mag)


    print(f"Cyclotron period: {T_c*1e6:.4f} μs")
    print(f"Theory Larmor radius: {r_L*1e3:.4f} mm")

    t_vv, r_vv, v_vv, KE_vv = velocity_verlet(B, v0, dt, t_final)
    t_boris, r_boris, v_boris, KE_boris = boris_algorithm(B, v0, dt, t_final)

    fig, axes = plt.subplots(2, 3, figsize=(15, 10))

    # Trajectory
    axes[0, 0].plot(r_vv[:,0]*1e3, r_vv[:,1]*1e3, 'b-',
                    label='Velocity Verlet', linewidth=2, alpha=0.8)
    axes[0, 0].plot(r_boris[:,0]*1e3, r_boris[:,1]*1e3, 'r--',
                    label='Boris', linewidth=2, alpha=0.8)
    circle = plt.Circle((0, 0), r_L*1e3, fill=False, color='k',
                        linestyle=':', linewidth=2, label=f'Theory: {r_L*1e3:.2f} mm')
    axes[0, 0].add_patch(circle)
    axes[0, 0].set_xlabel(r'$x$ (mm)')
    axes[0, 0].set_ylabel(r'$y$ (mm)')
    axes[0, 0].set_title(r'Trajectory ($xy$ plane)')
    axes[0, 0].legend()
    axes[0, 0].grid(True, alpha=0.3)
    axes[0, 0].axis('equal')

    # Energy
    error_vv = (KE_vv - KE_vv[0]) / KE_vv[0] * 100
    error_boris = (KE_boris - KE_boris[0]) / KE_boris[0] * 100

    axes[0, 1].plot(t_vv*1e6, error_vv, 'b-', label='Velocity Verlet', linewidth=2)
    axes[0, 1].plot(t_boris*1e6, error_boris, 'r--', label='Boris', linewidth=2)
    axes[0, 1].axhline(0, color='k', linestyle='--', alpha=0.5)
    axes[0, 1].set_xlabel(r'Time ($\mu$s)')
    axes[0, 1].set_ylabel(r'$\Delta KE/KE_0$ (\%)')
    axes[0, 1].set_title(r'Energy Conservation')
    axes[0, 1].legend()
    axes[0, 1].grid(True, alpha=0.3)
```

```python

    # Velocity space
    axes[0, 2].plot(v_vv[:,0]/1e5, v_vv[:,1]/1e5, 'b-',
                    label='Velocity Verlet', linewidth=2, alpha=0.8)
    axes[0, 2].plot(v_boris[:,0]/1e5, v_boris[:,1]/1e5, 'r--',
                    label='Boris', linewidth=2, alpha=0.8)
    axes[0, 2].set_xlabel(r'$v_x$ ($\times 10^5$ m/s)')
    axes[0, 2].set_ylabel(r'$v_y$ ($\times 10^5$ m/s)')
    axes[0, 2].set_title(r'Velocity Space')
    axes[0, 2].legend()
    axes[0, 2].grid(True, alpha=0.3)
    axes[0, 2].axis('equal')

    # Orbit radius
    r_vv_mag = np.linalg.norm(r_vv[:,:2], axis=1)
    r_boris_mag = np.linalg.norm(r_boris[:,:2], axis=1)

    axes[1, 0].plot(t_vv*1e6, r_vv_mag*1e3, 'b-', label='Velocity Verlet', linewidth=2)
    axes[1, 0].plot(t_boris*1e6, r_boris_mag*1e3, 'r--', label='Boris', linewidth=2)
    axes[1, 0].axhline(r_L*1e3, color='k', linestyle=':', label=f'Theory', linewidth=2)
    axes[1, 0].set_xlabel(r'Time ($\mu$s)')
    axes[1, 0].set_ylabel(r'Radius (mm)')
    axes[1, 0].set_title(r'Orbit Radius')
    axes[1, 0].legend()
    axes[1, 0].grid(True, alpha=0.3)

    # Speed
    speed_vv = np.linalg.norm(v_vv, axis=1)
    speed_boris = np.linalg.norm(v_boris, axis=1)

    axes[1, 1].plot(t_vv*1e6, speed_vv/1e5, 'b-', label='Velocity Verlet', linewidth=2)
    axes[1, 1].plot(t_boris*1e6, speed_boris/1e5, 'r--', label='Boris', linewidth=2)
    axes[1, 1].axhline(v0/1e5, color='k', linestyle=':', label='Initial', linewidth=2)
    axes[1, 1].set_xlabel(r'Time ($\mu$s)')
    axes[1, 1].set_ylabel(r'Speed ($\times 10^5$ m/s)')
    axes[1, 1].set_title(r'Particle Speed')
    axes[1, 1].legend()
    axes[1, 1].grid(True, alpha=0.3)

    # Phase error
    axes[1, 2].plot(t_vv*1e6, error_vv, 'b-', label='Velocity Verlet', linewidth=2.5)
    axes[1, 2].plot(t_boris*1e6, error_boris, 'r--', label='Boris', linewidth=2.5)
    axes[1, 2].set_xlabel(r'Time ($\mu$s)')
    axes[1, 2].set_ylabel(r'Energy Error (\%)')
    axes[1, 2].set_title(r'Relative Error')
    axes[1, 2].legend()
```

```python
    axes[1, 2].grid(True, alpha=0.3)

    plt.tight_layout()

    print(f"\nVelocity Verlet:")
    print(f"  Final radius: {r_vv_mag[-1]*1e3:.4f} mm")
    print(f"  Energy error: {error_vv[-1]:.2e}%")
    print(f"\nBoris:")
    print(f"  Final radius: {r_boris_mag[-1]*1e3:.4f} mm")
    print(f"  Energy error: {error_boris[-1]:.2e}%")

    return fig



def plot_exb_drift(B_mag=50e-6, E_mag=1e3, v0=1e5, t_final=5e-6):

    E = np.array([E_mag, 0.0, 0.0])
    B = np.array([0.0, 0.0, B_mag])
    v_initial = np.array([0.0, v0, 0.0])

    drift_velocity = np.cross(E, B) / np.dot(B, B)

    omega_c = abs(q_e) * B_mag / m_e
    T_c = 2 * np.pi / omega_c
    dt = T_c / 100


    print(f"Drift velocity (theory): {drift_velocity[0]:.3e} m/s")

    t_vv, pos_vv, vel_vv = velocity_verlet_fields(B, E, v_initial, dt, t_final)
    t_boris, pos_boris, vel_boris = boris_fields(B, E, v_initial, dt, t_final)

    fig, axes = plt.subplots(2, 2, figsize=(14, 10))

    # Trajectory
    axes[0, 0].plot(pos_vv[:,0]*1e6, pos_vv[:,1]*1e3, 'b-',
                    label='Velocity Verlet', linewidth=2, alpha=0.8)
    axes[0, 0].plot(pos_boris[:,0]*1e6, pos_boris[:,1]*1e3, 'r--',
                    label='Boris', linewidth=2, alpha=0.8)
    axes[0, 0].set_xlabel(r'$x$ drift ($\mu$m)')
    axes[0, 0].set_ylabel(r'$y$ gyration (mm)')
    axes[0, 0].set_title(r'Cycloid Trajectory')
    axes[0, 0].legend()
    axes[0, 0].grid(True, alpha=0.3)

    # Velocity Verlet components
    axes[0, 1].plot(t_vv*1e6, vel_vv[:,0], 'b-', label=r'$v_x$ (drift)', linewidth=1.5)
    axes[0, 1].plot(t_vv*1e6, vel_vv[:,1], 'b--', label=r'$v_y$ (gyration)', linewidth=1.5, alpha=0.8)
    axes[0, 1].axhline(drift_velocity[0], color='r', linestyle=':', label='Theory', linewidth=2)
```

```python
    axes[0, 1].set_xlabel(r'Time ($\mu$s)')
    axes[0, 1].set_ylabel(r'Velocity (m/s)')
    axes[0, 1].set_title(r'Velocity Verlet: Components')
    axes[0, 1].legend()
    axes[0, 1].grid(True, alpha=0.3)

    # Boris components
    axes[1, 0].plot(t_boris*1e6, vel_boris[:,0], 'r-', label=r'$v_x$ (drift)', linewidth=1.5)
    axes[1, 0].plot(t_boris*1e6, vel_boris[:,1], 'r--', label=r'$v_y$ (gyration)', linewidth=1.5, alpha=0.8)
    axes[1, 0].axhline(drift_velocity[0], color='k', linestyle=':', label='Theory', linewidth=2)
    axes[1, 0].set_xlabel(r'Time ($\mu$s)')
    axes[1, 0].set_ylabel(r'Velocity (m/s)')
    axes[1, 0].set_title(r'Boris: Components')
    axes[1, 0].legend()
    axes[1, 0].grid(True, alpha=0.3)

    # X-displacement
    axes[1, 1].plot(t_vv*1e6, pos_vv[:,0]*1e6, 'b-', label='Velocity Verlet', linewidth=2)
    axes[1, 1].plot(t_boris*1e6, pos_boris[:,0]*1e6, 'r--', label='Boris', linewidth=2)
    axes[1, 1].set_xlabel(r'Time ($\mu$s)')
    axes[1, 1].set_ylabel(r'$x$ displacement ($\mu$m)')
    axes[1, 1].set_title(r'Drift Displacement')
    axes[1, 1].legend()
    axes[1, 1].grid(True, alpha=0.3)

    plt.tight_layout()

    print(f"\nVelocity Verlet drift: {(pos_vv[-1,0] - pos_vv[0,0]) / t_vv[-1]:.3e} m/s")
    print(f"Boris drift: {(pos_boris[-1,0] - pos_boris[0,0]) / t_boris[-1]:.3e} m/s")

    return fig



def plot_trapped_particles():


    # Electron
    L_shell = 2.5
    r0 = L_shell * RE
    pos_e = np.array([r0, 0, 0])

    E_keV = 10.0
    E_J = E_keV * 1.602e-16
    v_total = np.sqrt(2 * E_J / m_e)

    pitch_angle = 60.0
```

```python
    v_perp = v_total * np.sin(np.radians(pitch_angle))
    v_para = v_total * np.cos(np.radians(pitch_angle))
    vel_e = np.array([0, v_perp, v_para])

    B0 = dipole_field(pos_e)
    B0_mag = np.linalg.norm(B0)
    omega_c = abs(q_e) * B0_mag / m_e
    T_gyro = 2 * np.pi / omega_c
    dt = T_gyro / 100
    t_final = 50 * T_gyro

    print(f"Electron: L={L_shell} RE, E={E_keV} keV, $\alpha ={pitch_angle}$")
    print(f"Gyroperiod: {T_gyro*1e6:.3f} μs")

    print("Running Velocity Verlet...")
    t_vv, pos_vv, vel_vv, B_vv = simulate_dipole_verlet(m_e, q_e, pos_e, vel_e, dt, t_final)

    print("Running Boris...")
    t_boris, pos_boris, vel_boris, B_boris = simulate_dipole_boris(m_e, q_e, pos_e, vel_e, dt, t_final)

    if len(t_vv) < 10 or len(t_boris) < 10:
        print("Warning: Particle escaped")
        return None

    def calculate_invariants(pos, vel, B):
        r_mag = np.linalg.norm(pos, axis=1)
        B_mag = np.linalg.norm(B, axis=1)
        v_mag = np.linalg.norm(vel, axis=1)

        v_perp_arr = np.array([np.linalg.norm(np.cross(vel[i], B[i]/B_mag[i]))
                               for i in range(len(vel))])
        mu = m_e * v_perp_arr**2 / (2 * B_mag)
        KE = 0.5 * m_e * v_mag**2

        return r_mag, mu, KE

    r_vv_mag, mu_vv, KE_vv = calculate_invariants(pos_vv, vel_vv, B_vv)
    r_boris_mag, mu_boris, KE_boris = calculate_invariants(pos_boris, vel_boris, B_boris)

    fig = plt.figure(figsize=(16, 12))

    # 3D
    ax1 = fig.add_subplot(2, 3, 1, projection='3d')
    ax1.plot(pos_vv[:,0]/RE, pos_vv[:,1]/RE, pos_vv[:,2]/RE, 'b-', linewidth=0.8, alpha=0.8)
    ax1.plot(pos_boris[:,0]/RE, pos_boris[:,1]/RE, pos_boris[:,2]/RE, 'r--', linewidth=0.8, alpha=0.8)
    ax1.set_xlabel(r'$x$ ($R_E$)')
    ax1.set_ylabel(r'$y$ ($R_E$)')
    ax1.set_zlabel(r'$z$ ($R_E$)')
    ax1.set_title(r'3D Trajectory')
```

```python

    # XY
    ax2 = fig.add_subplot(2, 3, 2)
    ax2.plot(pos_vv[:,0]/RE, pos_vv[:,1]/RE, 'b-', linewidth=0.8, alpha=0.8, label='Velocity Verlet')
    ax2.plot(pos_boris[:,0]/RE, pos_boris[:,1]/RE, 'r--', linewidth=0.8, alpha=0.8, label='Boris')
    circle = plt.Circle((0,0), 1, color='cyan', alpha=0.3)
    ax2.add_patch(circle)
    ax2.set_xlabel(r'$x$ ($R_E$)')
    ax2.set_ylabel(r'$y$ ($R_E$)')
    ax2.set_title(r'Equatorial Plane (Drift)')
    ax2.legend()
    ax2.axis('equal')
    ax2.grid(True, alpha=0.3)

    # XZ
    ax3 = fig.add_subplot(2, 3, 3)
    ax3.plot(pos_vv[:,0]/RE, pos_vv[:,2]/RE, 'b-', linewidth=0.8, alpha=0.8, label='Velocity Verlet')
    ax3.plot(pos_boris[:,0]/RE, pos_boris[:,2]/RE, 'r--', linewidth=0.8, alpha=0.8, label='Boris')
    circle = plt.Circle((0,0), 1, color='cyan', alpha=0.3)
    ax3.add_patch(circle)
    ax3.set_xlabel(r'$x$ ($R_E$)')
    ax3.set_ylabel(r'$z$ ($R_E$)')
    ax3.set_title(r'Meridional Plane (Bounce)')
    ax3.legend()
    ax3.axis('equal')
    ax3.grid(True, alpha=0.3)

    #ax3 = fig.add_subplot(2, 3, 3)
    #ax3.plot(pos_vv[:,0]/RE, pos_vv[:,2]/RE, 'b-', linewidth=2, alpha=0.9, label='Velocity Verlet')
    #ax3.plot(pos_boris[:,0]/RE, pos_boris[:,2]/RE, 'r--', linewidth=2, alpha=0.9, label='Boris')

    #ax3.set_xlabel(r'$x$ ($R_E$)', fontsize=9)
    #ax3.set_ylabel(r'$z$ ($R_E$)', fontsize=9)
    #ax3.set_title(r'Meridional Plane (Bounce Detail)', fontsize=10, fontweight='bold')

    # Extreme zoom to see bounce oscillations
    #z_max = np.max([np.max(np.abs(pos_vv[:,2])), np.max(np.abs(pos_boris[:,2]))]) / RE
    #x_min = np.min([np.min(pos_vv[:,0]), np.min(pos_boris[:,0])]) / RE
    #x_max = np.max([np.max(pos_vv[:,0]), np.max(pos_boris[:,0])]) / RE

    #ax3.set_xlim([x_min - 0.05, x_max + 0.05])
    #ax3.set_ylim([-z_max*1.3, z_max*1.3])

    #ax3.legend(fontsize=8, loc='upper right')
    #ax3.grid(True, alpha=0.3, linewidth=0.5)

```

```python

    # Magnetic moment
    ax4 = fig.add_subplot(2, 3, 4)
    ax4.plot(t_vv*1e3, (mu_vv - mu_vv[0])/mu_vv[0]*100, 'b-', linewidth=2, label='Velocity Verlet')
    ax4.plot(t_boris*1e3, (mu_boris - mu_boris[0])/mu_boris[0]*100, 'r--', linewidth=2, label='Boris')
    ax4.axhline(0, color='k', linestyle='--', alpha=0.5)
    ax4.set_xlabel(r'Time (ms)')
    ax4.set_ylabel(r'$\Delta \mu/\mu_0$ (\%)')
    ax4.set_title(r'Magnetic Moment (1st Invariant)')
    ax4.legend()
    ax4.grid(True, alpha=0.3)

    # Energy
    ax5 = fig.add_subplot(2, 3, 5)
    ax5.plot(t_vv*1e3, (KE_vv - KE_vv[0])/KE_vv[0]*100, 'b-', linewidth=2, label='Velocity Verlet')
    ax5.plot(t_boris*1e3, (KE_boris - KE_boris[0])/KE_boris[0]*100, 'r--', linewidth=2, label='Boris')
    ax5.axhline(0, color='k', linestyle='--', alpha=0.5)
    ax5.set_xlabel(r'Time (ms)')
    ax5.set_ylabel(r'$\Delta E/E_0$ (\%)')
    ax5.set_title(r'Kinetic Energy Conservation')
    ax5.legend()
    ax5.grid(True, alpha=0.3)

    # L-shell
    ax6 = fig.add_subplot(2, 3, 6)
    ax6.plot(t_vv*1e3, r_vv_mag/RE, 'b-', linewidth=1.5, label='Velocity Verlet')
    ax6.plot(t_boris*1e3, r_boris_mag/RE, 'r--', linewidth=1.5, label='Boris')
    ax6.axhline(L_shell, color='k', linestyle=':', alpha=0.5, label=f'$L={L_shell}$')
    ax6.set_xlabel(r'Time (ms)')
    ax6.set_ylabel(r'$L$-shell ($R_E$)')
    ax6.set_title(r'Radial Distance')
    ax6.legend()
    ax6.grid(True, alpha=0.3)



    plt.tight_layout()

    print(f"\nVelocity Verlet:")
    print(f"  $\mu$ variation: {(mu_vv[-1]-mu_vv[0])/mu_vv[0]*100:.4f}%")
    print(f"  E variation: {(KE_vv[-1]-KE_vv[0])/KE_vv[0]*100:.6f}%")

    print(f"\nBoris:")
    print(f"  $\mu$ variation: {(mu_boris[-1]-mu_boris[0])/mu_boris[0]*100:.4f}%")
    print(f"  E variation: {(KE_boris[-1]-KE_boris[0])/KE_boris[0]*100:.6f}%")

    return fig
```

```python
if __name__ == "__main__":

    fig1 = plot_cyclotron_motion()
    plt.savefig('01_cyclotron_verlet_vs_boris.png', dpi=150, bbox_inches='tight')

    fig3 = plot_exb_drift()
    plt.savefig('03_exb_drift_verlet_vs_boris.png', dpi=150, bbox_inches='tight')

    fig4 = plot_trapped_particles()
    if fig4:
        plt.savefig('04_trapped_verlet_vs_boris.png', dpi=150, bbox_inches='tight')

    plt.show()
```